

5.3 opencode ✓ plan small  
— build mode

## Lecture 7: Ensemble Methods

Vegard Høghaug Larsen

GRA4160: Predictive modelling with machine learning

February 18th 2026

# Plan for today:

- ▶ Introduction to ensemble methods
- ▶ Bagging
- ▶ Boosting
- ▶ Recap decision trees
- ▶ Random forests
- ▶ Extra trees

- ▶ A family of techniques that involve training multiple models and combining their predictions to form a more accurate and robust model
- ▶ Can be used with a variety of models (e.g. linear models, decision trees, or neural networks)
- ▶ By combining multiple models, the ensemble typically reduces variance and often boosts performance
- ▶ **A trade-off:** ensembles are often more powerful, but can be less interpretable than single models

# Types of Ensemble Methods

- ▶ **Bagging** (Bootstrap Aggregating)
  - ▶ Examples: Random Forests, Extra Trees
  - ▶ Reduces variance by training multiple models on **bootstrap samples** and averaging predictions

# Types of Ensemble Methods

- ▶ **Bagging** (Bootstrap Aggregating)

- ▶ Examples: Random Forests, Extra Trees
- ▶ Reduces variance by training multiple models on **bootstrap samples** and averaging predictions

- ▶ **Boosting** (Sequential Learning)

- ▶ Examples: Gradient Boosting, AdaBoost, XGBoost
- ▶ Trains models sequentially, focusing on misclassified examples to reduce bias

# Types of Ensemble Methods

- ▶ **Bagging** (Bootstrap Aggregating)
  - ▶ Examples: Random Forests, Extra Trees
  - ▶ Reduces variance by training multiple models on **bootstrap samples** and averaging predictions
- ▶ **Boosting** (Sequential Learning)
  - ▶ Examples: Gradient Boosting, AdaBoost, XGBoost
  - ▶ Trains models sequentially, focusing on misclassified examples to reduce bias
- ▶ **Bayesian Model Averaging** (BMA)
  - ▶ Trains multiple models and assigns weights based on posterior probabilities
  - ▶ Can dynamically update model weights as new data becomes available
  - ▶ Useful when model uncertainty needs to be explicitly accounted for

# Types of Ensemble Methods

- ▶ **Bagging** (Bootstrap Aggregating)
  - ▶ Examples: Random Forests, Extra Trees
  - ▶ Reduces variance by training multiple models on **bootstrap samples** and averaging predictions
- ▶ **Boosting** (Sequential Learning)
  - ▶ Examples: Gradient Boosting, AdaBoost, XGBoost
  - ▶ Trains models sequentially, focusing on misclassified examples to reduce bias
- ▶ **Bayesian Model Averaging** (BMA)
  - ▶ Trains multiple models and assigns weights based on posterior probabilities
  - ▶ Can dynamically update model weights as new data becomes available
  - ▶ Useful when model uncertainty needs to be explicitly accounted for
- ▶ **Stacking** (Stacked Generalization)
  - ▶ Combines multiple models by training a meta-model on their outputs
  - ▶ Often uses diverse base models (e.g., decision trees, SVMs, neural networks) to leverage complementary strengths
  - ▶ Example: A neural network trained on outputs of logistic regression, SVM, and a random forest

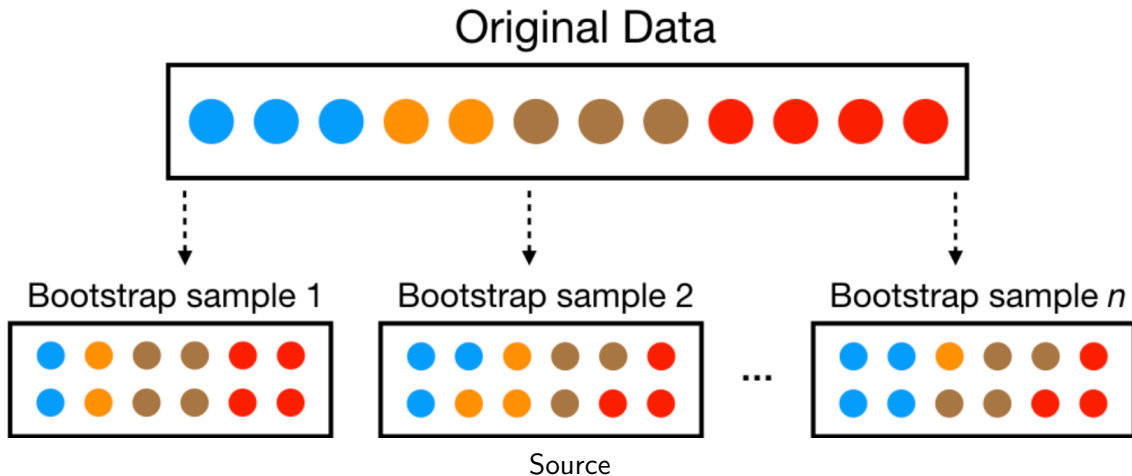
## Bagging

Creating Subsets of Data



- ▶ Bagging is short for **Bootstrap AGGregatING**
- ▶ Train multiple instances of a base model on *bootstrap* samples of the training data
- ▶ Combine their predictions (via majority vote or averaging) to form a final prediction
- ▶ Subsets are created using sampling with replacement
- ▶ Effective at reducing variance and mitigating overfitting for high-variance base learners (e.g., large decision trees)

- ▶ A resampling technique to approximate the sampling distribution of a statistic
- ▶ Repeatedly sample with replacement to create new datasets of the same size as the original
- ▶ About 63% of unique samples from the original dataset typically appear in each bootstrap sample
- ▶ Used to train multiple models (bagging) or provide performance estimates (e.g. OOB error)
- ▶ Especially useful for small datasets or when data is expensive to collect



# How does bagging work?

- ▶ Create multiple bootstrap samples of the training data
- ▶ Train a base model (e.g. a decision tree) on each bootstrap sample
- ▶ Use each trained model to make predictions
- ▶ Combine predictions from all models (majority vote for classification or averaging for regression)
- ▶ Final prediction often has lower variance and improved generalization

## Boosting

on a bit better than  
model is good as random guessing

- ▶ A technique that combines multiple *weak* learners sequentially
- ▶ Each subsequent model focuses more on samples that previous models misclassified (higher weight)
- ▶ Typically reduces bias, but can be prone to overfitting if not regularized
- ▶ Often uses simpler base learners (e.g. shallow trees), which are incrementally improved

# How does boosting work?

1. Initialize the weights of each training example equally
2. Train a base model on the training data using the current weights
3. Evaluate performance on the training data
4. Increase weights of misclassified examples and decrease weights of correctly classified examples
5. Repeat until stopping criterion (e.g. max iterations or minimal improvement)
6. Final prediction is a weighted combination of all models' predictions
7. **Learning rate** ( $\eta$ ) often controls the contribution of each weak learner

Cross Validate

- ▶ **AdaBoost:** Adaptive Boosting — one of the earliest boosting algorithms
- ▶ **XGBoost:** eXtreme Gradient Boosting — a popular gradient boosting framework
- ▶ Both typically use decision trees as base learners, though different objective and optimization strategies
- ▶ Widely used for tasks like image classification, ranking, and structured data problems



1. **Regularization:** includes L1 and L2 terms to prevent overfitting
2. **Tree pruning:** prunes during construction to improve generalization
3. **Handling missing values:** a built-in mechanism
4. **Cross-validation:** integrated for hyperparameter tuning
5. **Parallelization and out-of-core computation:** allows XGBoost to scale efficiently

- ▶ Built by recursively splitting data into subsets based on a feature threshold that provides the best information gain
- ▶ Leaf nodes contain final predictions (class labels or numeric values)
- ▶ Advantages: easy to interpret, handle mixed data types, minimal preprocessing
- ▶ Disadvantages: prone to overfitting if unpruned, can be unstable w.r.t. small data changes

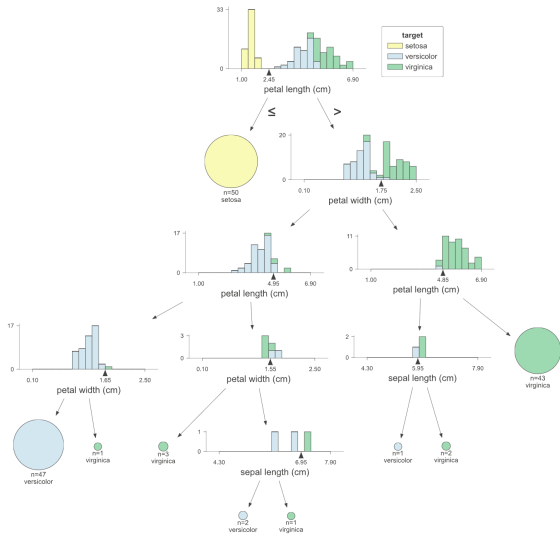
# Visualization of decision trees

<https://github.com/parrrt/dtreviz>

```
import dtreeviz # remember to pip install dtreeviz

viz = dtreeviz.model(clf, X, y,
                    target_name="target",
                    feature_names=iris.feature_names,
                    class_names=list(iris.target_names))
viz.view(scale=1.3)
```

## Visualization of decision trees



UB

- ▶ Bagging multiple decision trees alone can lead to highly correlated trees
- ▶ Random Forest adds **feature randomness** to reduce correlation among trees
- ▶ In addition to bootstrapping samples, each split considers only a random subset of features
- ▶ Reduces variance by decorrelating trees; improves predictive performance

1. Each tree is built from a bootstrapped subset of the training data
2. At each split, randomly select a set of predictors (e.g.  $\sqrt{p}$ ) out of  $p$
3. From these predictors, choose the best predictor and threshold
4. Result: lower correlation among trees and often higher accuracy

- ▶ Each decision tree in the ensemble is trained on a unique bootstrap sample
- ▶ Within each tree, only a subset of features is considered for splits
- ▶ Final classification prediction by majority vote; for regression, often averaged
- ▶ Provides good performance out-of-the-box and can handle large feature spaces

We 10 Random Forest is overfitting

- ▶ Stands for **Extremely Randomized Trees**
- ▶ Similar to Random Forest in that multiple trees are trained on bootstrap samples
- ▶ At each node, a random subset of features is chosen (like in RF)
- ▶ **Key difference:** Extra Trees picks *random* splitting thresholds, rather than searching for the optimal threshold

an ~~Extremely~~ large tree set



# Voting Classifier

1 & 2 models say you're dead, you are

$m_1$   $m_2$   $m_3$

20 60 55 10

↓  
model  
3 different base probabilities

Doesn't make sense to use ~~How~~ Voting when  
using 2 classifiers

Using  $K_C$ : Randomly Random Subsets  
size  $k$  features